Divide and Conquer Algorithm

Divide and Conquer algorithm is a problem-solving strategy that involves.

- Divide: Break the given problem into smaller non-overlapping problems.
- Conquer: Solve Smaller Problems
- Combine: Use the Solutions of Smaller Problems to find the overall result.

Examples of Divide and Conquer are Merge Sort, Quick Sort, Binary Search.

- There is no need of explicit combine step in some algorithms like Binary Search and Quick Sort. Although in Merge Sort, the combine step is the main step.
- The divide step can be trivial in some algorithms (like in Merge Sort and Binary Search, we simply divide in two equal halves). The divide step can be complex in some algorithms like Quick Sort.

Merge Sort:

Merge sort is a popular sorting algorithm known for its efficiency and stability. It follows the Divide and Conquer approach. It works by recursively dividing the input array into two halves, recursively sorting the two halves and finally merging them back together to obtain the sorted array.

Here's a step-by-step explanation of how merge sort works:

1. Divide: Divide the list or array recursively into two halves until it can no more be divided.
2. Conquer: Each subarray is sorted individually using the merge sort algorithm.
3. Merge: The sorted subarrays are merged back together in sorted order. The process continues until all elements from both subarrays have been merged.

Let's sort the array or list [38, 27, 43, 10] using Merge Sort

Let's look at the working of above example:

Divide:

- *[38, 27, 43, 10] is divided into [38, 27] and [43, 10] .*
- *[38, 27] is divided into [38] and [27] .*
- *[43, 10] is divided into [43] and [10] .*

Conquer:

- *[38] is already sorted.*
- *[27] is already sorted.*
- *[43] is already sorted.*

- *[10] is already sorted.*

Merge:

- *Merge [38] and [27] to get [27, 38] .*
- *Merge [43] and [10] to get [10,43] .*
- *Merge [27, 38] and [10,43] to get the final sorted list [10, 27, 38, 43]*

Therefore, the sorted list is [10, 27, 38, 43] .

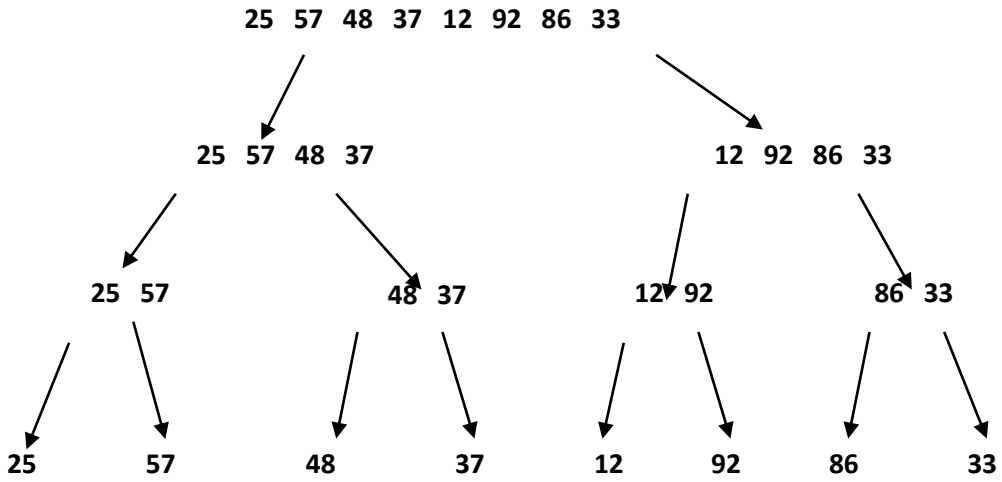
Algorithm outline

If size of list is greater than 1

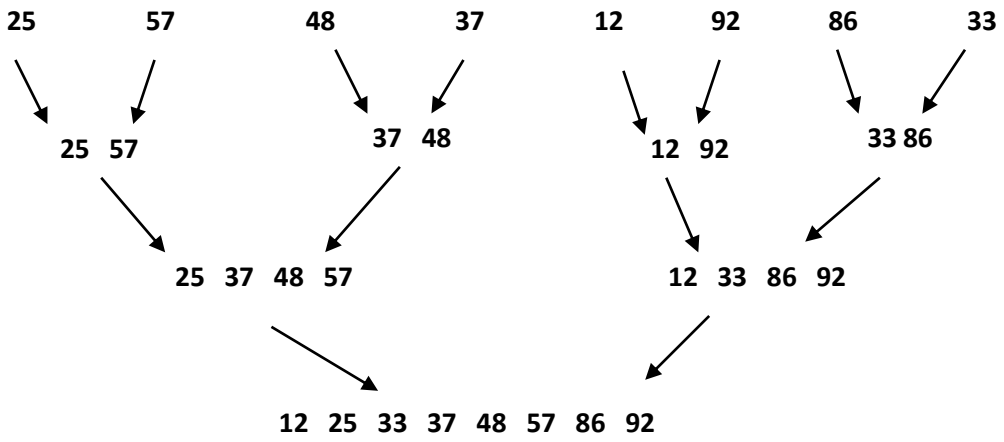
- divide the list into two sublists of sizes nearly equal as possible
- recursively sort the sublist separately.
- Merge the two sorted sublists into a single sorted list

End if

- **First Phase:** Partition the list in two equal halves, until the list size is 1



- **Second Phase:** Merge the sorted sublists



Merging description:

To simply, the algorithm merge to sorted sublist into a third list.

When finished, we copy back the third list in the original sorted halves to get the sorted list.

The basic merging algorithm takes two input sorted arrays A and B, and an output array C. We initialize the ponters Aptr, Bptr, and Cptr to point the beginning of their respective arrays. The smaller of A[Aptr] and B[Bptr] is copied to the next entry in C and appropriate pointers are advanced. When any one of the list has been finished, the remainder of the other list is copied to C.

Aptr

25	37	48	57

Bptr

12	33	86	92

Cptr

12			

Compare 25 and 12 and insert minimum(12) into array C and move Bptr and Cptr

Aptr

25	37	48	57

Bptr

12	33	86	92

Cptr

12				

Compare 25 and 33 and insert minimum(25) into array C and move Aptr and Cptr

Aptr

25	37	48	57

Bptr

12	33	86	92

Cptr

12	25			

Compare 37 and 33 and insert minimum (33) into array C and move Bptr and Cptr.

In this way the two lists are merged.

Quick Sort:

It is the fastest known sorting algorithms used in practice.

Basic idea

> Divide the list into two sublists such that all elements in the first list is less than some pivot key and all elements in the second list is greater than the pivot key,
>
> and finally sort the sublists independently and combine them.

Algorithm:

If size of list *A* is greater than 1

- Pick any element *v* from *A*. This is called the *pivot*
- Partition the list *A* by placing *v* in some position *j*, such that
  - all elements before position *j* are less than or equal to *v*

- all elements after position *j* are greater than or equal to *v*

- Recursively sort the sublists *A*[0] through *A*[*j*-1] and *A*[*j*+1] through *A*[*N*-1]
- Return *A*[0] through *A*[*j*-1] followed by *A*[*j*] (the pivot) followed by *A*[*j*+1] through *A*[*N*-1]

25 57 48 37 12 92 86 33

Choose the first element 25 as the pivot and partition the array

(12) 25 (57 48 37 92 86 33)

The first subarray is automatically sorted

12 25 (57 48 37 92 86 33)

Choose the first element 57 of the second subarray as the pivot and partition the subarray

12 25 (48 37 33) 57 (92 86)

12 25 (48 37 33) 57 (92 86)

12 25 (37 33) 48 57 (92 86)

12 25 (33) 37 48 57 (92 86)

12 25 33 37 48 57 (92 86)

12 25 33 37 48 57 (92 86)

12 25 33 37 48 57 (86) 92

12 25 33 37 48 57 86 92

Binary Search:

Binary search is an extremely efficient algorithm. This search technique searches the given item in minimum possible comparisons. To do the binary search, first we have to sort the array elements. The logic behind this technique is given below.

1. First find the middle element of the array.
2. Compare the middle element with an item.
3. There are three cases.
   a) If it is a desired element then search is successful,

   b) If it is less than the desired item then search only in the first half of the array.

c) If it is greater than the desired item, search in the second half of the array.

4. Repeat the same steps until an element is found or search area is exhausted.

In this way, at each step we reduce the length of the list to be searched by half.

Requirements

i. The list must be ordered
ii. Rapid random access is required, so we cannot use binary search for a linked list

Greedy Algorithm

Greedy is an algorithmic paradigm that builds up a solution piece by piece, always choosing the next piece that offers the most obvious and immediate benefit. Greedy algorithms are used for optimization problems.

An optimization problem can be solved using Greedy if the problem has the following property:

- At every step, we can make a choice that looks best at the moment, and we get the optimal solution to the complete problem.
- Some popular Greedy Algorithms are Kruskal's algorithm, Prim's Algorithm, Huffman coding and Dijkstra's algorithm.

How does the Greedy Algorithm works?

Greedy Algorithm solve optimization problems by making the best local choice at each step in the hope of finding the global optimum. It's like taking the best option available at each moment, hoping it will lead to the best overall outcome.

Here's how it works:

1. Start with the initial state of the problem. This is the starting point from where you begin making choices.
2. Evaluate all possible choices you can make from the current state. Consider all the options available at that specific moment.
3. Choose the option that seems best at that moment, regardless of future consequences. This is the "greedy" part - you take the best option available now, even if it might not be the best in the long run.
4. Move to the new state based on your chosen option. This becomes your new starting point for the next iteration.

5. Repeat steps 2-4 until you reach the goal state or no further progress is possible. Keep making the best local choices until you reach the end of the problem or get stuck.

Example:

Let's say you have a set of coins with values [1, 2, 5, 10] and you need to give minimum number of coin to someone change for 39.

The greedy algorithm for making change would work as follows:

- *Step-1: Start with the largest coin value that is less than or equal to the amount to be changed. In this case, the largest coin less than or equal to 39 is 10.*
- *Step- 2: Subtract the largest coin value from the amount to be changed, and add the coin to the solution. In this case, subtracting 10 from 39 gives 29, and we add one 10-coin to the solution.*

Repeat steps 1 and 2 until the amount to be changed becomes 0.

The greedy algorithm is not always the optimal solution for every optimization problem, as shown in the example below.

- When using the greedy approach to make change for the amount 20 with the coin denominations [18, 1, 10], the algorithm starts by selecting the largest coin value that is less than or equal to the target amount. In this case, the largest coin is 18, so the algorithm selects one 18 coin. After subtracting 18 from 20, the remaining amount is 2.
- At this point, the greedy algorithm chooses the next largest coin less than or equal to 2, which is 1. It then selects two 1 coins to make up the remaining amount. So, the greedy approach results in using one 18 coin and two 1 coins.

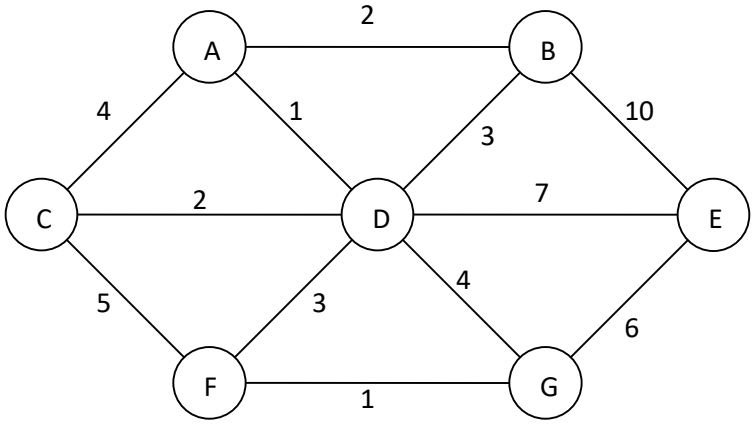
However, the greedy approach fails to find the optimal solution in this case. Although it uses three coins, a better solution would have been to use two 10 coins, resulting in a total of only two coins (10 + 10 = 20).

Kruskal's Algorithm:

To determine minimum spanning tree consider a graph G with n vertices.

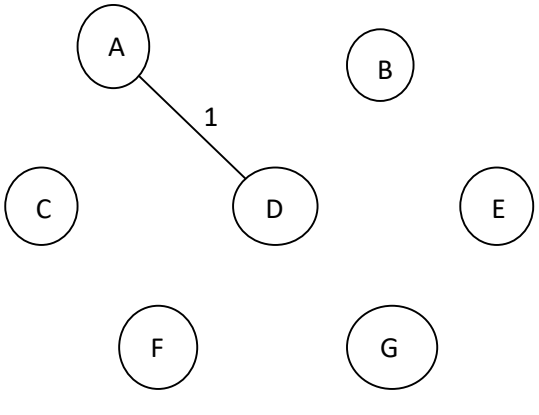
Step 1: List all the edges of the graph G with increasing weights
Step 2: Proceed sequentially to select one edge at a time joining n vertices of G such that no cycle is formed until n-1 edges are selected.

Step 3: Draw the n-1 edges that were selected forming a minimal spanning tree T of G.

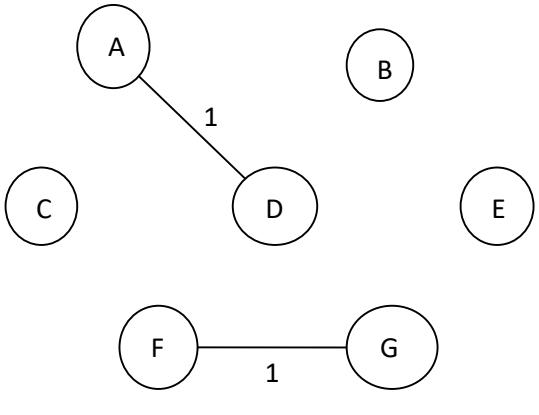


We have,

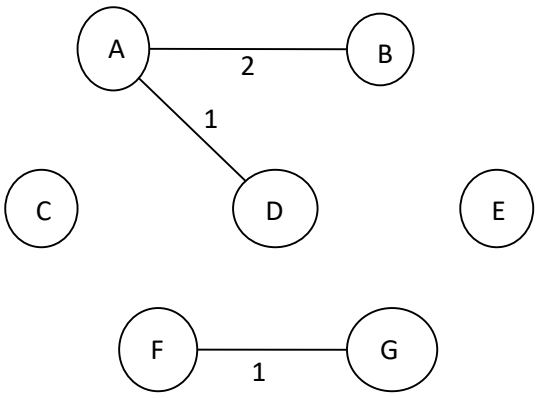
Edges:	AD	FG	AB	CD	BD	AC	DG	CF	EG	DE	DF	BE
Weights	1	1	2	2	3	4	4	5	6	7	8	10



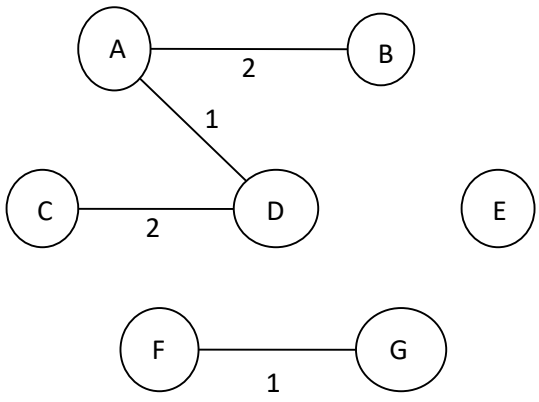
(1)



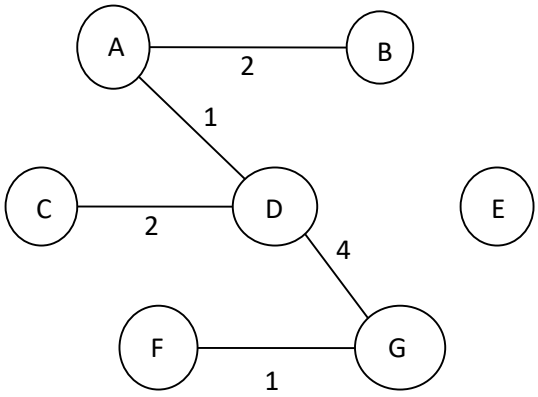
(2)



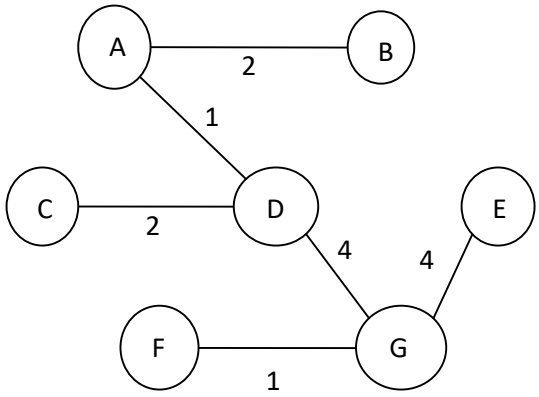
(3)



(4)



(5)



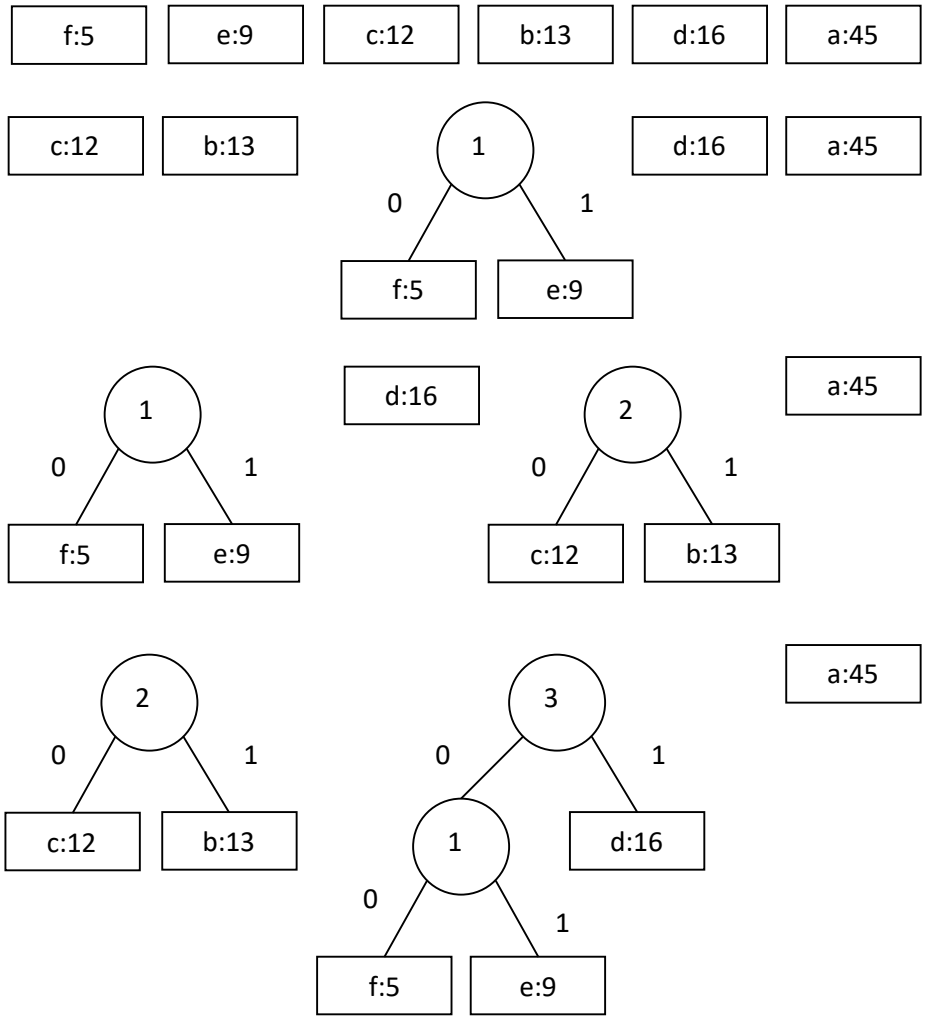
(6)

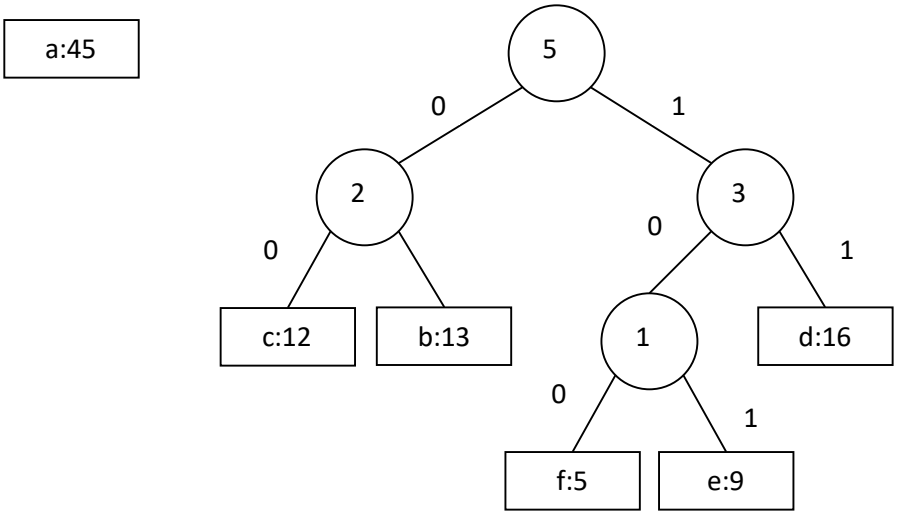
Hence, the minimum spanning tree is generated.

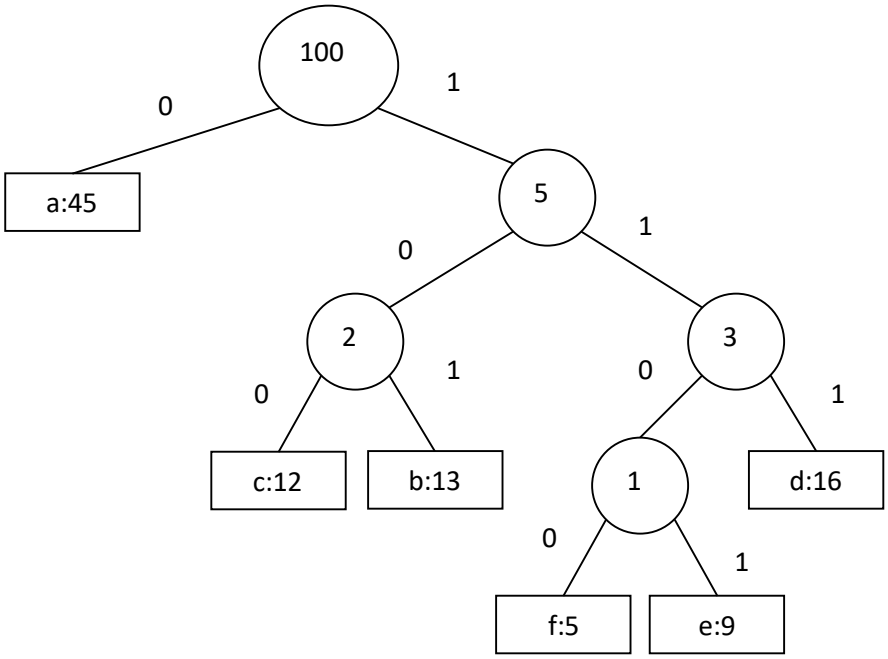
Huffman Coding:

Example: Construct a Huffman code for following symbols.

	a	b	C	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100


f:5
e:9
c:12
b:13
d:16
a:45
c:12
b:13
1
0
1
f:5
e:9
d:16
a:45
1
0
1
f:5
e:9
2
0
1
c:12
b:13
a:45
2
0
1
c:12
b:13
3
0
1
1
d:16
0
1
f:5
e:9





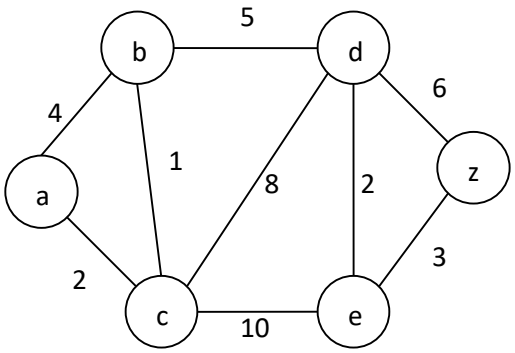
Each part shows the contents of the queue sorted into increasing order by frequency. At each step, the two trees with lowest frequencies are merged.

Leaves are shown as rectangles containing a character and its frequency. Internal nodes are shown as circles containing the sum of the frequencies of its children. An edge connecting an internal node with its children is labeled 0 if it is an edge to a lift child and 1 if it is an edge to a right child.

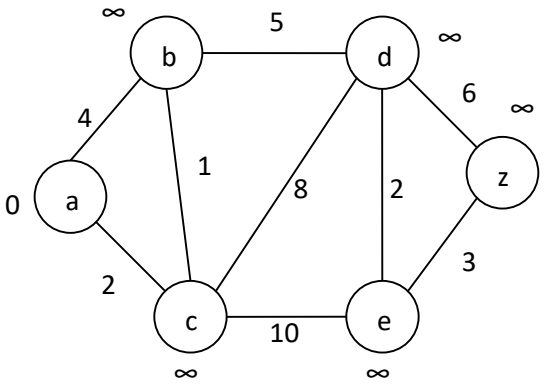
The codeword for a letter is the sequence of labels on the edges connecting the root to the leaf for that letter.

Dijkstra's Algorithm:

1. Mark all the vertices as unknown
2. for each vertex v keep a distance d_v from source vertex s to v initially set to infinity except for s which is set to $d_s = 0$
3. repeat these steps until all vertices are known
    i. select a vertex v, which has the smallest d_v among all the unknown vertices
    ii. mark v as known
    iii. for each vertex w adjacent to v
        i. if w is unknown and $d_v + \text{cost}(v, w) < d_w$
        update d_w to $d_v + \text{cost}(v, w)$



Given graph with weights



(a)

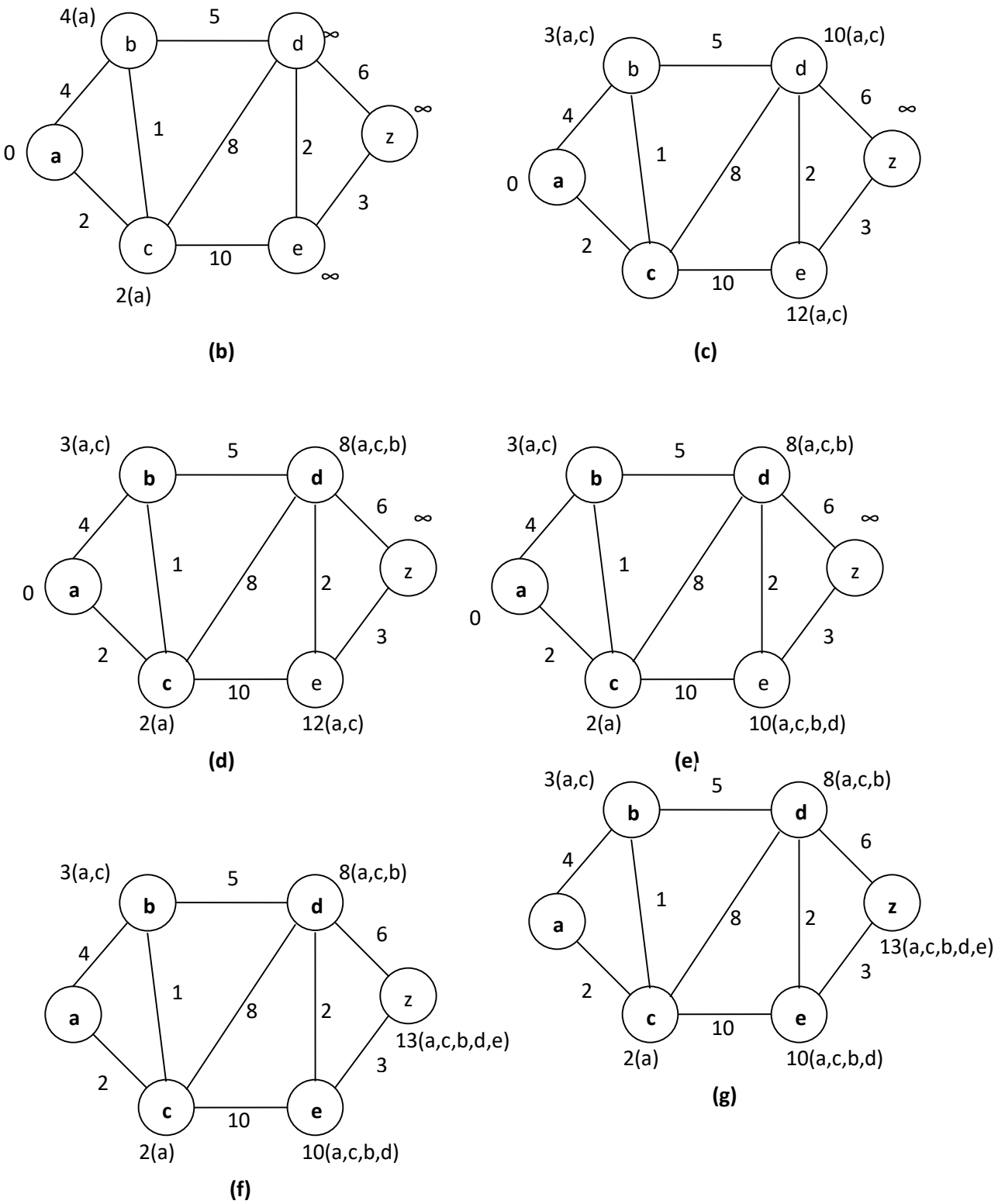


Fig: Using Dijkstra's Algorithm to find shortest path from a to z

Assignment 1:
Explain Backtracking Algorithms, Branch & Bound and Randomized Algorithms.

Time Complexity

Time Complexity measures how the running time of an algorithm increases with input size (n).

It does not measure actual time in seconds, but the number of basic operations.

Common Time Complexities

Complexity	Name	Example
O(1)	Constant	Access array element
O(log n)	Logarithmic	Binary search
O(n)	Linear	Linear search
O(n log n)	Linearithmic	Merge sort
$O(n^2)$	Quadratic	Bubble sort
$O(2^n)$	Exponential	Recursive Fibonacci

Example

```
for(i = 0; i < n; i++)
  printf("Hello");
```

Executes **n times → O(n)**

Space Complexity:

Space Complexity measures the extra memory used by an algorithm.

Includes:

- Variables
- Data structures
- Function call stack (recursion)

Example

```
int sum(int n){
  int s = 0;
  for(int i=1;i<=n;i++)
    s += i;
  return s;
}
```

Uses constant variables → **O(1)** space

Recursive Example

```
int fact(int n){
  if(n==0) return 1;
  return n * fact(n-1);
}
```

Recursion stack of size **n** → **O(n)** space

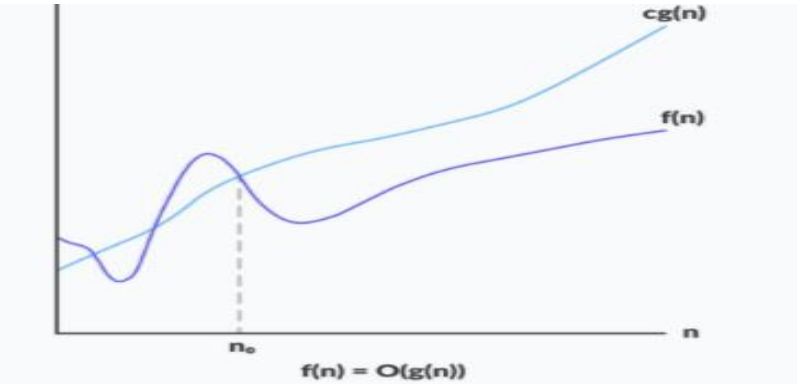
Asymptotic notations

Asymptotic notations describe algorithm behavior for very large input size ($n \to \infty$)**.**

Big O

This notation gives an upper bound for function to within constant factor.

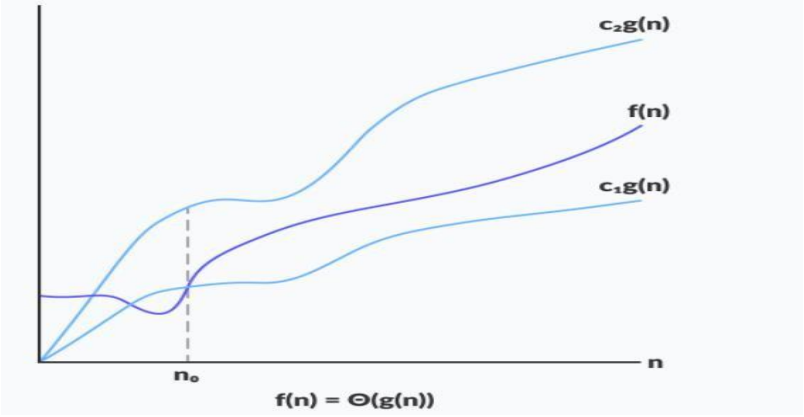
f(n) = O(g(n) if there exists positive constants n_0 and C such that to the right of n_0, the value of f(n) always lies on or below Cg(n).



Big Theta

Theta notation bounds a function within constant factors.

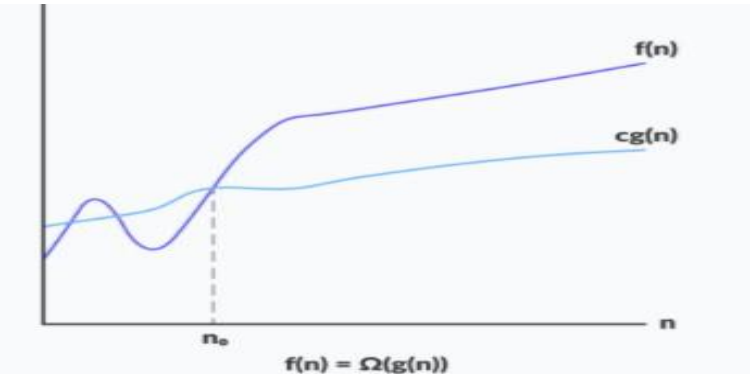
f(n)= Θ(g(n) if there exists positive constants n_0, C_1 and C_2 such that to the right of n_0, the value of f(n) always lies between $C_1g(n)$ and $C_2g(n)$.



Big Omega

This notation gives the lower bound for a function to within constant factor.

F(n)= Ω g(n) if there exists positive constants n_0 and C such that to the right of n_0 the value of f(n) always lie on or above Cg(n).



Big O Notation Examples

Big O notations are used to clarify running time function. If f(n) is O(g(n), then informally, f(n) is within a constant factor of g(n).

Definition: Let f(n) and g(n) be two functions on positive integers. We say f(n) is O(g(n)) if there exist two positive constants C and K such that f(n) <= C g(n) for all n >= K.

Example 1

f(n) = 10n+5 and g(n)= n
Solution:

Here, f(n) = O(g(n)) = O(n)

To show f(n) is O(g(n)), we must show constants C and k such that $f(n) \leq Cg(n)$ for all $n \geq k$.

Or, $10n+5 \leq Cn$ for all $n \geq k$.

We are allowed to choose C and k to be integer, we want as long as they are positive.
They can be as big as we want but they cannot be the function of n.

Let C=15, then we need to show, $10n+5 \leq$
15n Solving for n,

$10n+5 \leq$
15n Or,
$5 \leq 5n$
Or, $1 \leq n$

So, f(n)= 10+5 <= 15.g(n) for all $n \geq 1$.

Hence, we have shown f(n) is O(g(n)).